INDEX

NAME Prisha. T SUBJECT DAA

STD. IT DIV. D ROLL NO. 195 SCHOOL REC

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1		Finding the time Complexity of an Algorithm. Counter method		
1)	3	Problem 1	6/8	
2)	7	Problem 2	6/8	
3)	11	Problem 3	6/8	
4)	15	Problem 4	6/8	
5)	19	Problem 5	6/8	
		Divide and Conquer		
1)	23	Number of zeros in a given array	11/8	
2)	27	Majority element	11/8	
3)	31	Finding floor value	11/8	
4)	35	Two elements sum to x	11/8	
5)	39	Implementation of Quick sort	20/8	
		Greedy Algorithms		
1)	43	1-Gr-Coin Problem	27/08	
2)	47	2-Gr-Cookies Problem	27/08	
3)	51	3-Gr-Burger Problem	31/08	
4)	55	4-Gr-Array Summax problem	27/08	
5)	59	5-Gr-Product of Array element	28/08	
		Dynamic Programming		
1	63	1-DP-Playing with Numbers	24/09	
2	67	2-DP-Playing with chessboard	24/09	
3	71	3-DP longest Common Subsequence	24/09	
4	75	4-DP longest non-decreasing Subsequence	24/09	

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
		Competitive Programming		
1	79	1- Finding Duplicates - O(n²) TC, O(1) SC	24/9	
2	83	2- Finding Duplicates - O(n) TC, O(1) SC	24/9	
3	87	3- Print Intersection of 2 sorted arrays O(m*n) TC & O(1) Space complexity.	24/9	
4	91	4 - Print Intersection of 2 Sorted arrays - O(m+n) TC & O(1) SC	1/10	
5	95	5- Pair with Difference - O(n^2) TC & O(1) SC	1/10	
6	98	6 - Pairs with Difference - O(n) TC, O(1) SC	1/10	

Finding Time Complexity of the Algorithm

Aim :- To find the time complexity of an algorithm using counter method.

Algorithm :-

```
void function (int n)
{
    int i = 1;
    int s = 1;
    while (s <= n)
    {
        i++;
        s += i;
    }
}
```

Program :-

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int counter = 0;
    int i = 1;
    int s = 1;
    counter += 2;
    while (1) {
        counter++;
        if (s > n)
            break;
        i++;
        counter++;
        s += i;
        counter++;
    }
    printf("%d", counter);
    return 0;
}
```

Result :-

The algorithm has been converted into a program and the time complexity has been found using counter method.

Output :-

Input	Expected	Got
9	12	12
4	9	9

Finding Time Complexity of an Algorithm

Aim :-

To find the time complexity of an algorithm using counter method.

Algorithm :-

```
void func (int n)
{
   if (n == 1)
   {
      printf ("*");
   }
   else
   {
      for (int i = 1; i <= n; i++)
      {
         for (int j = 1; j <= n; j++)
         {
            printf ("*");
            printf ("*");
            break;
         }
      }
   }
}
```

Program :-

```
#include <stdio.h>
int main() {
    int n;
    Scanf ("%d", &n);
    int count;
    if (n==-1)
        count = 1;
    else
        count = 5 * n + 2;
    printf ("%d", count);
    return 0;
}
```

Result :-

The algorithm has been converted into a program and the time complexity has been found using counter method.

Output :-

Input	Expected	Got
2	12	12
1000	5002	5002
143	717	717

Finding Time Complexity of an Algorithm

Aim :-

To find the time complexity of an algorithm using counter method.

Algorithm :-

```
factor (num) {
    for (i = 1; i <= num; ++i)
    {
        if (num % i == 0)
        {
            printf("%d", i);
        }
    }
}
```

Result

The algorithm has been executed successfully and the Time complexity has been found using counter method.

Program :-

```
#include <stdio.h>
int main() {
 int num, i;
 int count = 0;
 Scanf ("%d", &num);
for (i = 1, count++; i <= num; ++i)
{
    count++;
    if (num%.i == = 0)
    {
        count++;
    }
Count++;
}
printf ("%d", count);
return 0;
}
```

Result :-

The algorithm has been converted into a program and the time complexity has been found using counter method.

Output :

Input	Expected	Got
12	31	31
25	54	54
4	12	12

Finding Time Complexity of an algorithm

Aim:-

To find the time complexity of an algorithm using counter method.

Algorithm:-

```
void function(int n)
{
  int c = 0;
  for(int i = n/2; i<n; i++)
    for(int j = 1; j<n; j = 2*j)
      for(int k=1; k<n; k = k*2)
        c++;
}
```

Program :-

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    int count = 1;
    int i, j, k;
    for (i = n/2 ; i<n; i++)
    {
        count++;
        for (j = 1; j<n; j = 2 * j)
        {
            count++;
            for (k=1; k<n; k = k * 2)
            {
                count++;
                count++;
            }
            count++;
        }
        count++;
    }
    count++;
    printf("%d", count);
    return 0;
}
```

Result :- The algorithm has been converted into a program and the time complexity has been found using counter method.

Output :-

Input	Expected	Got
4	30	30
10	212	212

Finding Time Complexity of an algorithm.

Aim :- To find the time complexity of an algorithm using counter method.

Algorithm :-

```
Void rev = 0, remainder;
while (n != 0)
{
    remainder = n % 10;
    rev = rev * 10 + remainder;
    n /= 10;
}
print (rev);
}
```

Program :-

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int c = 3;
    while(n != 0)
    {
        c = c + 4;
        n = n/10;
    }
    printf("%d", c);
}
```

Result :-

The algorithm has been converted into a program and the time complexity has been found using counter method.

Output :-

Input	Expected	Got
12	11	11
1234	19	19

Divide and Conquer

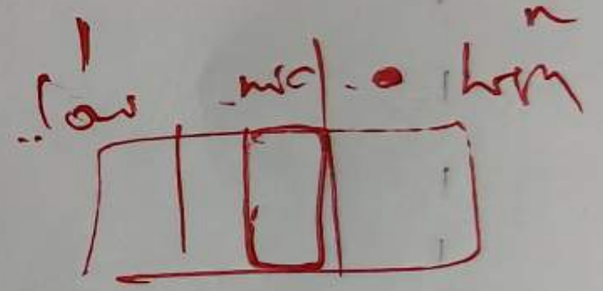
(Number of zeros in a given array)

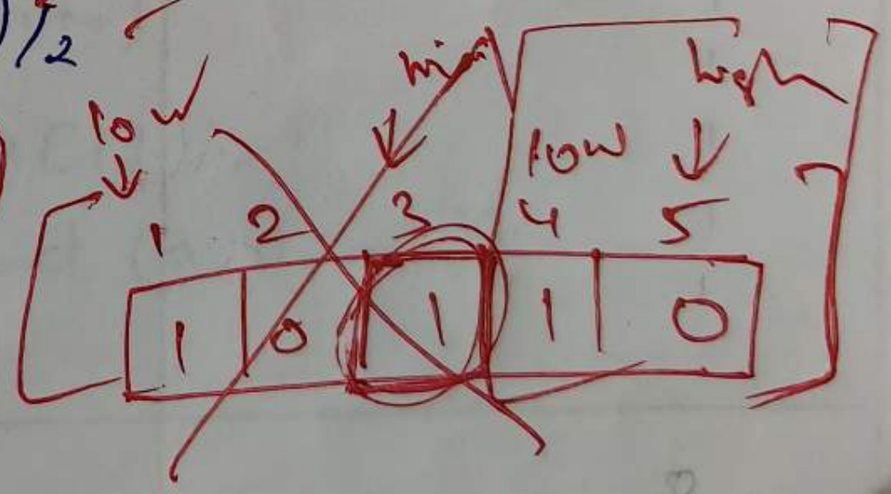
Aim :-

To write a program using the Divide and Conquer method to count the number of 0s in an array where all 1s appear first followed by 0s.

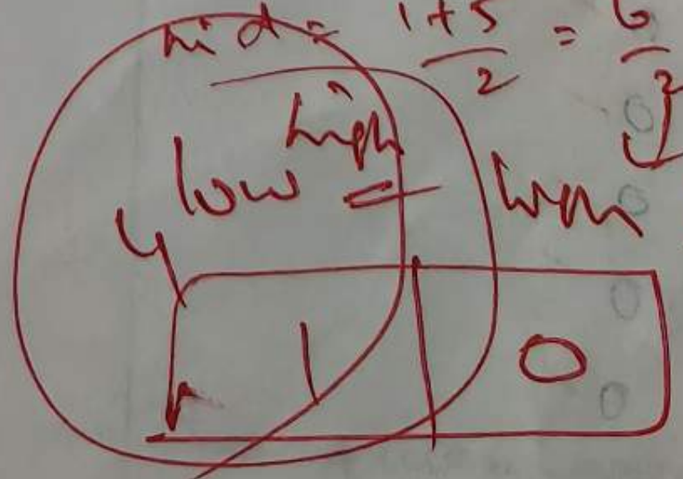
Algorithm :-

```
Algorithm div_and_con ()
{
    Read m and arr
    low = 0, high = m-1
    WHILE low <= high
        mid = (low + high)/2
        IF arr[mid] = 1
            low = mid + 1
        ELSE
            high = mid-1
    END WHILE
    PRINT m - low
}
```





while (l <= 5)



4 <= 5

Algorithm:

```
Algorithm div_and_con()
{
   Read m and arr
   low = 1 , high = m
WHILE low <= high
   mid = (low + high) /2
   IF arr[mid] = 1
   low = mid + 1
   ELSE IF (l = = h)
      return(a[l] = = 0)
   ELSE
      high = mid - 1
END WHILE
   PRINT m - low
}
```

Program:-

```
#include <stdio.h>
int count(int a[], int l, int r) {
    if (l > r)
        return 0;
    if (l == r)
        return (a[l] == 0);
    if (a[l] == 0)
        return r-l+1;
    int mid = (l+r)/2;
    return count(a, l, mid) + count(a, mid+1, r);
}
int main() {
    int m;
    scanf("%d", &m);
    int a[m];
    for(int i = 0; i<m; i++)
        scanf("%d", &a[i]);
    printf("%d\n", count(a, 0, m-1));
    return 0;
}
```

Result:-

Thus the number of 0s in the given array was found successfully using the Divide and Conquer method.

Output :-

Input	Expected	Got
5 1 1 1 0 0	2	2
10 1 1 1 1 1 1 1 1 1 1	0	0
8 0 0 0 0 0 0 0 0 0	8	8

Divide and Conquer

(Majority Element)

2) **Aim:-**

To find the majority element in an array using the Boyer - Moore Voting Algorithm.

Algorithm :

```
Algorithm majority_element ()
{
    Read n
    count = 0
    FOR each element x
        IF count = 0, candidate = x
        IF x = candidate, count ++
        ELSE count - -
    END FOR
    Print candidate
}
```

Program :-

```
#include <stdio.h>
int main() {
    int n, x, count = 0, candidate = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        if (count == 0) candidate = x;
        count += (x == candidate) ? 1 : -1;
    }
    printf("%d\n", candidate);
    return 0;
}
```

Result :-

Thus the majority element has been found and printed successfully.

Output :-

Input	Expected	Got
3	3	3
3 2 3		
1 1 1		

Divide and Conquer

(Finding Floor Value)

3) **Aim :-**

To find the floor value of a given number x in a sorted array using the Divide and Conquer method.

Algorithm :-

```
Algorithm floor-value ()
{
    Read n, array and x
    low = 1 , high = n
    WHILE low <= high
        mid = (low + high) / 2
        IF a[mid] <= x, floor = a[mid], low = mid+1
        ELSE high = mid - 1
    END WHILE
    Print floor
}
```

Program:

```
#include <stdio.h>
int floor(int a[], int l, int r, int x) {
if (l > r) return -1;
int m = (l + r) / 2;
if (a[m] == x) return a[m];
if (a[m] > x) return floor(a, l, m-1, x);
int f = floor(a, m+1, r, x);
return f == -1 ? a[m] : f;
}

int main() {
    int n, x;
    scanf("%d", &n);
    int a[n];
    for(int i = 0; i<n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &x);
    printf("%d\n", floor(a, 0, n-1, x));
    return 0;
}
```

Result:-

The floor value of the given number x is found and printed successfully.

Input	Expected	Got
[6]	2	
1.		
2		2
8		
1.0		
1.2		
1.9		
[5]		
[5]		
10		
22		
85	85	85
108		
129		
[106]		

Divide and Conquer
(Two elements sum to x)

1) **Aim :-**

To find whether two elements in a sorted array have a sum equal to x using the Divide and Conquer method.

Algorithm :-

```
Algorithm equal_sum ()
{
    Read n, array and x
    Set left = 0, right = n - 1
    REPEAT
        IF a[left] + a[right] = x
            Print both elements and STOP
        IF sum > x, right --
        ELSE left ++
    UNTIL left >= right
    Print "No"
}
```

Program :-

```
#include <stdio.h>
int find(int a[], int l, int r, int x, int *p, int
                                                *q)
{
    if (l >= r) return 0;
    if (a[l] + a[r] == x) {
        *p = a[l]; *q = a[r];
        return 1;
    }
    if (a[l] + a[r] > x)
        return find(a, l, r-1, x, p, q);
    return find(a, l+1, r, x, p, q);
}
int main() {
    int n, x, p, q;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    scanf("%d", &x);
    if (find(a, 0, n-1, x, &p, &q))
        printf("%d\n%d\n", p, q);
    else
        printf("No\n");
    return 0;
}
```

Input	Expected	Got
4 [2 4 8 10] 14	4 10	4 10
5 [2 4 6 8 10] 100	No	No

Result :-

Thus the program successfully finds and prints the two elements whose sum equals 'x' otherwise, it prints "No".

(Implementation of Quick Sort)

5) **Aim :-**

To write a program which implements the quick sort algorithm.

Algorithm :-

```
Algorithm Quick_sort ()
{
    Sort (A, low, high)
    IF low < high
        P = divide (A, low, high)
        Sort (A, low, position - 1)
        Sort (A, p+1, high)
Divide (A, low, high)
    choose last element as key
    i = low - 1
    FOR j = low To high - 1
        IF A[j] < Key
            i = i + 1
            SWAP A[i] and A[j]
    SWAP A[i+1] and A[high]
    Return i+1
```

Program :-

```
#include <stdio.h>
void q(int a[], int l, int r) {
 int i = l, j = r, p = a[(l+r)/2], t;
 while(i<=j) {
   while (a[i]<p) i++;
   whil (a[j] >p) j--;
   if (i<=j) {
     t = a[i]; a[i]=a[j]; a[j]=t;
     i++; j--;
   }
 }
 if (l<j) q(a, l, j);
 if (i<r) q(a, i, r);
}
int main() {
  int n, a[100];
  scanf("%d", &n);
  for(int i=0; i<n; i++) scanf("%d", &a[i]);
  q(a, 0, n-1);
  for(int i=0; i<n; i++) printf("%d ", a[i]);
}
```

Result :-

Thus, the Quick sort algorithm was successfully implemented.

Output:-

Input	Expected	Output
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98
10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114

0/10

(3) Greedy Algorithm
(1-Gr-Coin Problem)

1) Aim :-

To write a C program to find the minimum number of coins required to make a given amount.

Algorithm :-

```
Algorithm greedy ()
{
   Read V
   count = 0
   denominations = [1000,500,100,50,20,10,5,2,1]
   FOR each denomination d
      count = count + V/d
      V = V % d
   END FOR
   PRINT count
```

Program :-

```
#include <stdio.h>
int main() {
  int v, count = 0;
  int a[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
  scanf("%d", &v);
  for (int i = 0; i < 9; i++) {
    count += v / a[i];
    v %= a[i];
  }
  printf("%d", count);
  return 0;
}
```

Result :-

Thus, the program successfully finds the minimum number of coins / notes required to make the given amount.

Input	Expected	Got
49	5	5
64	4	4

Greedy Algorithm

(2-G- Cookies Problem)

2) Aim :-

To write a program to find the maximum number of children who can be happy by assigning suitable cookies.

Algorithm :-

```
Algorithm greedy()
{
  Read number of children
  Read greed values
  Read number of cookies
  Read cookie sizes

  Sort children and cookies

  i = 0
  j = 0
  count = 0

  WHILE i < children AND j < cookies
    IF cookie[j] >= greed[i]
      count = count + 1
      i = i + 1
    END IF
    j = j + 1
  END WHILE

  Print Count
}
```

Program :-

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}
int main() {
    int n, m, i = 0, j = 0, count = 0;
    scanf("%d", &n);
    int g[n];
    for(i=0; i<n; i++) scanf("%d", &g[i]);
    scanf("%d", &m);
    int s[m];
    for(i = 0; i<m; i++) scanf("%d", &s[i]);
    qsort(g, n, sizeof(int), cmp);
    qsort(s, m, sizeof(int), cmp);
    i = j = 0;
    while (i<n && j<m) {
        if (s[j] >= g[i]) {
            count++;
            i++;
        }
        j++;
    }
    printf("%d", count);
    return 0;
}
```

Result:-

Thus, the program successfully finds the maximum number of content children by assigning each child a suitable cookie.

Output :-

Input	Expected	Got
2 1 2 3 1 2 3	2	2

Greedy Algorithm

3-G - Burger Problem

3) Aim:- To write a program to determine the min distance a person needs to run after eating a number of burgers.

Algorithm:-

```
Algorithm mincal () {
    Start
    Read n and calories array arr[0..n-1]
    Sort the array in descending order.
    Initialize sum← 0
    For i← 0 to i← n-1 Do
        Sum← Sum+ (n*i * arr[i])
    Print Sum
    Stop
}
```

Program

```
#include <math.h>
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++) scanf("%d", &arr[i]);
    int temp;
    for (int i=0; i<n; i++) {
        for (int j=i+1; j<n; j++) {
            if (arr[i] < arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
```

```
int sum = 0
for(int i = 0; i<n; i++) {
    sum += (pow(n,i) * arr[i]);
}
printf("%d", sum);
    return 0;
}
```

Result :-

Thus the program was written and executed successfully.

Output :-

Input	Expected	Got
3 1 3 2	18	18
4 7496	389	389
3 5107	76	76

Greedy Algorithm

Array Sum Max Problem

Problem :- To write a program to maximize the sum of arr[i] * and its corresponding index.

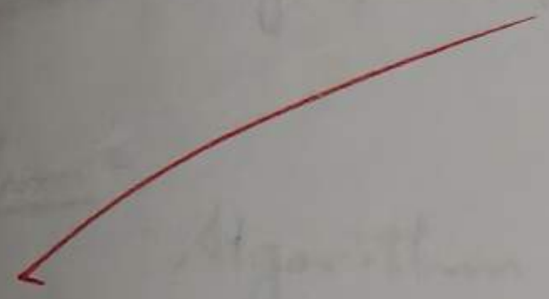
Algorithm :-

Algorithm maxsum() {
Start
read n and array a[0...n-1]
Sort a[] in ascending order using qsort.
Initialize sum=0
For i=0 to n-1 do
Sum = sum + (a[i]*i)
print sum
Stop
}

Program :-

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a const void *b) {
    return *(int*a) - *(int*b);
}
int main() {
int n;
scanf("%d",&n);
int a[n];
for(inti = 0;i<n;i++) scanf("%d",&a[i]),
qsort(a,n,sizeof(int),cmp);
int sum=0;
```

```
for (int i = 0; i<n; i++) sum += a[i]*k
printf("%d", sum);
return 0;
}
```



Result :-

Thus the program was written and executed successfully.

Output :-

Input	Expected	Got
5 2 5 3 4 0 2 45 3	40 45	40 45

Greedy Algorithm

Product of array elements - Minimum sum

Aim: To write a program to arrange two array such that sum of pairwise products is minimized.

Algorithm:

```
Algorithm minsum() {
    Start
    read n, array A[0...n-1] and B[0...n-1]
    Sort A is ascending order
    sort B in descending order
    Initialize sum = 0
    for i: 0 to n-1 do
    sum← sum+(A[i]*B[i])
    print sum
    Stop
}
```

Program:-

```
#include <stdio.h>
#include <stdlib.h>
int compare1(const void *a, const void *b){
    return (*(int*) a - *(int*) b);
}
int compare2 (const void * a, const void * b){
    return (*(int*) b - *(int*) a);
}
int main() {
  int n; sum = 0;
  scanf("%d", &n);
  int a[n], b[n];
for (int i = 0; i<n; i++) scanf("%d", &a[i]);
for (int i = 0; i<n; i++) scanf("%d", &b[i]);
```

```
qsort ( a, n, size of ( int), compare1 );
qsort (b, n, sizeof (int), compare2 );
for (int i =0 ; i<n ; i++) {
        sum+ = (a[i] * b[i]);
}
printf ("%d", sum);
return 0;
}
```

Result :

Thus the program was written and executed successfully.

Output :-

Input	Expected	Got
3 1 2 3 4 5 6 4 7 5 1 2 1 3 4 1	28 590	28 590

Dynamic Programming

1 - DP - Playing with Numbers

Aim :- To find the number of possible ways to represent a positive integer 'n' using only the numbers 1 and 3 using Dynamic Programming.

Algorithm :-

```
Algorithm numbers() {
Start
Read n
Set dp[0] = 1
For i = 1 to n
  dp[i] = dp[i-1]
  If i >= 3
     dp[i] = dp[i] + dp[i-3]
  End IF
End FOR
Print dp[n]
Stop
}
```

Program :-

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    long long dp[n+1];
    dp[0] = 1;
    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i-1];
        if (i >= 3)
            dp[i] += dp[i-3];
    printf("%lld", dp[n]);
    return 0;
}
```

Result :-

Thus the program was successfully executed to find the number of possible ways to represent a given positive integer n using 1 and 3.

Output :-

Input	Expected	Got
6	6	6
25	8641	8641
100	24382819596721629	24382819596721629

Dynamic Programming

2- DP- Playing with Chessboard

Aim :- To find the maximum monetary value path from the top-left cell (0,0) to the bottom right cell (n-1, n-1) using Dynamic Programming, where movement is allowed only right or down.

Algorithm :-

```
Algorithm Chessboard () {
    Start
    Read n
    Read the n x n matrix
    Create dp[n][n]
    dp[0][0] = board[0][0]
    For i = 1 to n-1
        dp[i][0] = dp[i-1][0] + board[i][0]
    For i = 1 to n-1
        For j = 1 to n-1
            dp[i][j] = board[i][j] + maximum
                (dp[i-1][j], dp[i][j-1])
    Print dp[n-1][n-1]
    Stop
}
```

Program:-

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int a[n][n], dp[n][n];
    for (int i = 0; i<n; i++)
        for (int j = 0; j<n; j++)
            scanf("%d", &a[i][j]);
    dp[0][0] = a[0][0];
    for (int i = 1; i<n; i++)
        dp[i][0] = dp[i-1][0] + a[i][0];
    for (int j = 1; j<n; j++)
        dp[0][j] = dp[0][j-1] + a[0][j];
    for (int i = 1; i<n; i++) {
        for (int j = 1; j<n; j++) {
            int max = dp[i-1][j] > dp[i][j-1];
            dp[i-1][j] : dp[i][j-1];
            dp[i][j] = a[i][j] + max;
        }
    }
    printf("%d", dp[n-1][n-1]);
    return 0;
}
```

Result:-

Thus, the maximum monetary value path from the top-left cell to the bottom right cell was successfully found using dynamic Programming.

Output :-

Input	Executed	Got
3 1 2 4 2 3 4 8 7 1	19	19
3 1 3 1 1 5 1 4 2 1	12	12

Dynamic Programming

3 - Dp - Longest Common Subsequence

Aim:- To find the length of the Longest Common Subsequence (LCS) between two strings using Dynamic Programming. The common subsequence need not be contiguous.

Algorithm:-

Algorithm Subsequence () {
Read string S1 & S2
m = length of S1
n = length of S2
Create dp[m+1][n+1]
Set first row and first column to 0
For i = 1 to m
    For j = 1 to n
        If S1[i-1] == S2[j-1]
            dp[i][j] = dp[i-1][j-1] + 1
        Else
            dp[i][j] = maximum(dp[i-1][j],
                               dp[i][j-1])
        End If
Print dp[m][n]
Stop

Program :-

```
#include<stdio.h>
#include<string.h>
int main() {
    char s1[100], s2[100];
    scanf("%s", s1);
    scanf("%s", s2);
    int m = strlen(s1);
    int n = strlen(s2);
    int dp[m+1][n+1];
    for(int i = 0; i<=m; i++)
        dp[i][0] = 0;
    for(int j = 0; j<=n; j++)
        dp[0][j] = 0;
    for(int i=1; i<=m; i++) {
        for(int j=1; j<=m; j++) {
            if(s1[i-1] == s2[j-1])
                dp[i][j] = dp[i-1][j-1]+1;
            else
                dp[i][j] = dp[i-1][j] > dp[i][j-1]
                    ? dp[i-1][j] : dp[i][j-1];
        }
    }
    printf("%d", dp[m][m]);
    return 0;
}
```

Result :-

Thus, the length of the Longest Common Subsequence between the given two strings was successfully found using dynamic Programming.

Input	Expected	Got
a a b	2	2
a z b		
A B C D		
A B C D	4	4

Dynamic Programming

4-DP- Longest non-decreasing Subsequence

Aim: To find the length of the Longest Non-Decreasing Subsequence (LNDS) in a given sequence using Dynamic Programming.

Algorithm:

```
Algorithm NDS_Subsequence () {
Start
Read n
Read array a
For i = 0 to n-1
    dp[i] = 1
For i = 1 to n-1
    For j = 0 to i-1
        If a[j] <= a[i]
            dp[i] = max (dp[i], dp[j]+1)
max = 1
For j = 0 to n-1
    max = max (max, dp[i])
Print max
Stop
}
```

Program :-

```
#include <stdio.h>
int main() {
    int n, a[100], dp[100], max = 1;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        dp[i] = 1;
    }
    for (int i = 1; i<n; i++)
        for (int j = 0; j<i; j++)
            if (a[j] <= a[i] && dp[i] < dp[j])
                dp[i] = dp[j] + 1;
    for (int i = 0; i<n; i++)
        if (dp[i] > max) max = dp[i];
    printf("%d", max);
    return 0;
}
```

Result :-

Thus, The length of the longest Non-Decreasing Subsequence was successfully found using dynamic programing.

Output :-

Input	Expected	Got
9 -1 3 4 5 2 2 2 2 3	6	6
7 1 2 2 4 5 7 6	6	6

Competative Programming

1- Finding Duplicates -O(n²) Time Complexity + O(1) Space Complexity.

Aim :- To find the duplicate element in an array containing integers from 1 to n.

Algorithm :-

```
Algorithm duplicates () {
Start
Read n
Read array a
For i = 0 to n-1
    For j = i+1 to n-1
        If a[i] == a[j]
            Print a[i]
            Stop
Stop
}
```

Program :-

```
#include <stdio.h>
int main () {
    int n, a[100];
    scanf("%d", &n);
    for(int i = 0; i<n; i++)
        scanf("%d", &a[i]);
    for(int i = 0; i<n; i++)
    for(int j = i+1; j<n ; j++)
        if (a[i] == [j]){
            printf("%d", a[i]);
            return 0;
        }
}return 0;
```

Result :-

Thus, the duplicate element in the given array was successfully found & displayed.

Output :-

Input	Expected	Got
5 1 2 3 4 4	4	4
5 1 1 2 3 4	1	1
11 10 9 7 6 5 1 2 3 8 4	7	7

Competative Programming

2- Finding Duplicates - O(n) TimeComplexity, O(1) Space Complexity

Aim :- find the To, duplicate element in a given array of integers from 1 to n.

Algorithm :-

Algorithm Duplicates () {
Start
Read n
Read array a
For i = 0 to n-1
For j = i+1 to n-1
If a[i] = a[j]
Print a[i]
Stop
End If
End for
End for
Stop

Program :-

```
#include <stdio.h>
int main() {
    int n, a[100];
    Scanf("%d", &n);
for (int i = 0; i<n; i++) scanf("%d", &a[i]);
for (int i = 0; i<n; i++)
    for (int j = i+1 ; j<n ; j++)
        if (a [i] == a [j]) {
            printf ("%d", a[i]);
            return 0;
        }
}
```

Result :-

Thus, the duplicate element in the given array was successfully found and displayed.

Output :-

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4	1	1

3- Print Intersection of 2 sorted arrays - O(m+n) Time Complexity, O(1) Space Complexity.

Aim :-

The Aim is to find all the elements which occur in both the arrays.

Algorithm :-

Start

Read T

For each test case:

Read m and array A

Read n and array B

For i = 0 to m-1:

For j = 0 to n-1:

If A[i] == B[j]:

Print A[i]

Break

End.

Program :-

```
#include <stdio.h>
int main() {
 int T, m, n, i, j;
 scanf("%d", &T);
while (T--) {
   scanf("%d", &m);
   int a[m];
for (i=0; i<m; i++)
    scanf("%d", &a[i]);
scanf("%d", &n);
int b[n];
for (i=0; i<n; i++) {
   for(j=0; j<n; j++) {
    if (a[i] == b[j]){
        printf("%d ", a[i]);
        break;
 } } }
 printf("\n");
}
    return 0;
}
```

Result :-

Thus the program to find all the elements

Output :-

Input	Expected	Got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57
1 6 1 2 3 4 5 6 2 16	16	16

4. Print Intersection of 2 sorted arrays - O(m+n) Time Complexity, O(1) Space Complexity.

Aim :-

To find all the elements which occur in both the arrays. (Using Pointers)

Algorithm :-

Start

Read T

For each test case:

```
int main() {
int T, m, n, i, j;
scanf("%d", &T);
while (T--) {
    scanf("%d", &m);
    int a[m];
    for (i=0; i<m; i++)
        scanf("%d", &a[i]);
scanf("%d", &n);
int b[n];
for (i=0; i<n; i++)
    scanf("%d", &b[i]);
i = j = 0;
while (i<m && j<n) {
if (a[i] == b[j]) {
    printf("%d", a[i]);
    i++;
    j++;
}
else if (a[i] < b[j])
    i++;
else
    j++;
}
printf("\n");
}
return 0;
}
```

Result :-

Thus the program to find all the elements in both the arrays has been executed successfully

Output :-

Input	Expected	Got
1 => No. of Test cases (T) 3 10 17 57 6 2 7 10 15 57 24 6 1 6 1 2 3 4 5 6 2 1 6	10 57 1 6	10 57 1 6

Competative Programming

5 - Pair with Difference - O(n^2) Time Complexity, O(1) Space Complexity

Aim :-

To find whether there exists a pair of elements in a sorted array whose difference is equal to the given non-negative integer K.

Algorithm :-

Start
Read n
Read array A
Read k

For i = 0 to n-1
    For j = i+1 to n-1 :
        If A[j] - A[i] == k :
            Print 1
            Stop

Print 0

END

Program :-

```
#include <stdio.h>
int main() {
    int n, k, i, j;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &k);
    for (i = 0; i < n; i++) {
        for (j = i+1; j < n; j++) {
            if (a[j] - a[i] == k) {
                printf("1");
                return 0;
            }
        }
    }
    printf("0");
    return 0;
}
```

Result :-

Thus, the program successfully finds whether a pair with given difference exists.

Output :-

Input	Expected	Got
3 1 3 5 4	1	1
10 1 4 6 8 12 14 15 20 21 25 1	1	1
10 1 2 3 5 11 14 16 24 28 29 0	0	0

Competative Programming

6-Pair with Difference - O(n) Time Complexity, O(1) Space Complexity

Aim :-

To find whether there exists a pair of elements in a sorted array whose difference is equal to the given non-negative integer k, using O(n) time & O(1) extra space.

Algorithm :-

Start
Read n
Read array A
Read k
Set i = 0 & j = 1
while i < n AND j < n:
If i == j:
j++
Else IF A[j] - A[i] == k:
Print 1
Stop
ELSE IF A[j] - A[i] < k:
j++
ELSE
i++
Print 0
END

, O(1) Space

elements

qual

ng O(n)

Programs :-

```
#include <stdio.h>
int main() {
    int n, k, i = 0, j = 1;
    scanf("%d", &n);
    int a[n];
    for (int x = 0; x < n; x++)
        scanf("%d", &a[x]);
    scanf("%d", &k);
    while (i < n && j < n) {
        if (i == j) {
            j++;
        }
        else if (a[j] - a[i] == k) {
            printf("1");
            return 0;
        }
        else if (a[j] - a[i] < k) {
            j++;
        }
        else {
            i++;
        }
    }
    printf("0");
    return 0;
}
```

Input	Expected	Got
3 1 3 5 4	1	1
10 1 4 6 8 12 14 15 20 21 25 1	1	1
10 1 2 3 5 11 14 16 24 28 29 0	0	0

Result :-

Thus, the program successfully checks whether a pair with the given difference exists using $O(n)$ time and $O(1)$ extra space.